Project 2: MultiDoc Evidence Agent: stage plan

A controlled LangGraph agent for multi-document review, built on Project 1

New repository: multidoc-evidence-agent (future project)

Weeks 8 to 12 (24 Dec 2026 to 27 Jan 2027) · about 17 of 76 project hours

Target roles: Applied AI Engineer, Agentic AI Engineer, Forward Deployed Engineer

Prepared for: Shalin Vachheta

Status: plan v1.2 for a FUTURE project, deferred until Project 1 is finished. Nothing is built. Week numbers inside are provisional
Targets marked TBD are set from the first runs. Companion: Project 1 plan (separate PDF) and docs/TENDER_RAG_REFEREN
Version 1.2 · 4 November 2026

Contents

1. Summary and status 3
2. Problem and use case 3
3. How it connects to Project 1 4
4. Job targeting for this project 5
5. Interview-question map 5
6. Agents and permissions 7
7. State, memory, tools and outputs 7
8. Data: planted facts, answer keys from the generator 7
9. Task set (15 tasks) 8
10. Evaluation and metrics 8
11. Safety, approval and authorization 9
12. Stage overview and schedule 10
13. Stage P2-1: LangGraph basics 11
14. Stage P2-2: Data and single agent 12
15. Stage P2-3: MCP link 12
16. Stage P2-4: LangSmith evaluation 12
17. Stage P2-5: Safety tests 13
18. Stage P2-6: Supervisor experiment (stretch) 13
19. Mandatory, stretch, risks and cut order 14
20. Deliverables, evidence and interview stories 14
21. Decisions, next steps and sources 15

PART 1 · CONCEPT AND TARGETING

What the agent does, how it builds on Project 1, which jobs it answers

1. Summary and status

Deferred. This project starts after Project 1 is finished. The week numbers, the week-by-week table and the timeline in this PDF come from the earlier combined schedule and are provisional; the stage contents and hours (5, 5, 2, 2, 3) stay valid.

Decision. Build one bounded LangGraph agent that compares two to four contracts and policies, finds conflicts and missing clauses with sources, pauses for human approval, and only then exports a cited report. It uses Project 1's search as its tool, over MCP. A single agent comes first; a supervisor version is an experiment that is kept only if it is measurably better. About 17 of the 76 project hours.

Item	Value
Repository	multidoc-evidence-agent (new; created in week 8)
What it is	Evidence-first document review: retrieve, extract typed fields, compare, flag conflicts and gaps, verify sources, ask for approval, export
Roles it targets	Applied AI Engineer, Agentic AI Engineer (2 to 3 years, stretch), Forward Deployed Engineer / AI Consultant; also strengthens junior AI Engineer applications
Core evidence	Task-success rate, tool correctness, field-extraction F1, conflict recall, approval enforced, injection test, loop guard, execution traces
Stack	Python, LangGraph, LangChain components, Pydantic, MCP, LangSmith, Docker, pytest
Hours	P2-1 5 · P2-2 5 · P2-3 2 · P2-4 2 · P2-5 3 = 17 h; stretch experiment P2-6 has no planned hours
Depends on	Project 1 stages S1 and S2 (fixed retrieval, evaluated) and the access-control behaviour in S3; contract v1 in the Project 1 reference, section 20

Honest status. This is a future project. Nothing exists yet: no repository, no generator, no agent. Every number here is a design target, not a result. Local small models are weak at tool calling and structured output; the data is synthetic, so a hosted model is an allowed fallback for this project only, and must be labelled in the README.

What is new in version 1.2

- Separate PDF for this project; the Project 1 plan and reference no longer carry agent details beyond the interface contract.
- From the interview question sets: a plain-Python agent drill before LangGraph; a loop guard; a post-condition check that the export really happened; idempotent export; field-extraction scores; explicit supersession and ambiguous cases in the data.
- From the review: supersession needs evidence (newer date alone is not enough); the supervisor is a stretch experiment; synthetic answer keys come from the generator, never from an LLM.

2. Problem and use case

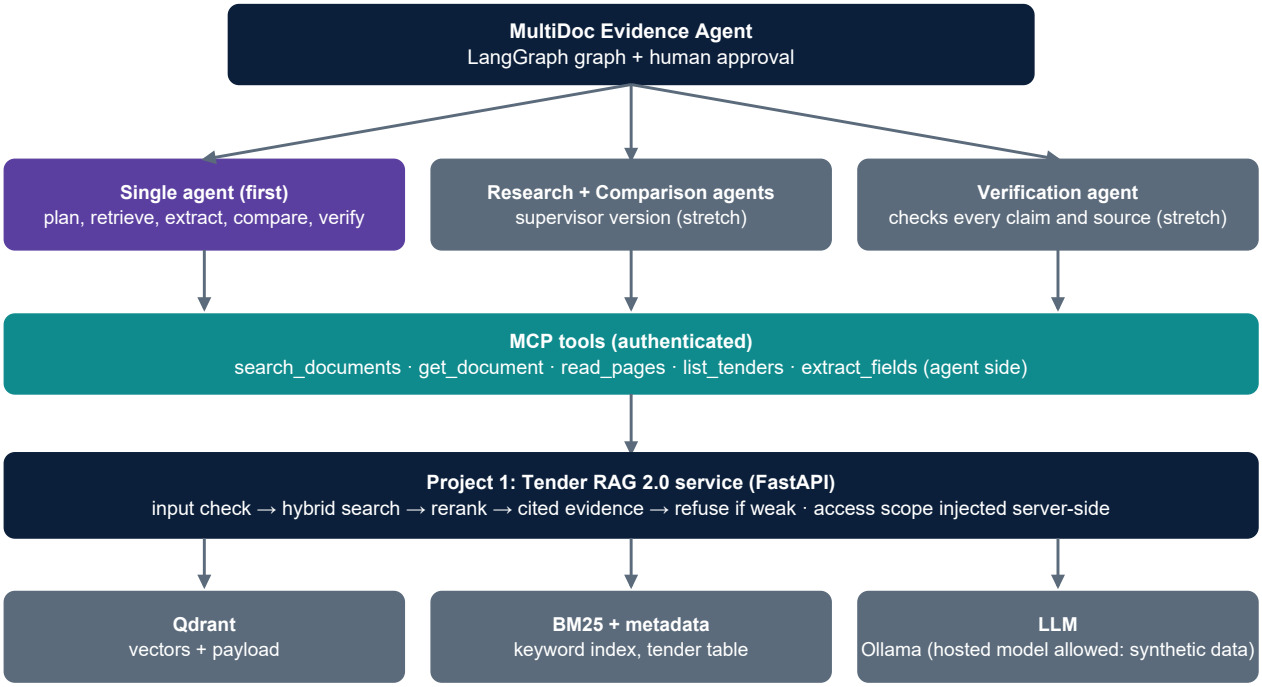
Organisations hold contracts, policies and amendments with overlapping, missing or contradictory information. Manual comparison is slow, and a conventional chatbot cannot reliably complete the whole review. The research behind this: the best model in EnterpriseRAG met single constraints 83.8 % of the time but all constraints together only 26.8 %, and flagged contradictions in under 45 % of cases; in HERB the best agentic method averaged 32.96, with retrieval as the main bottleneck.

Example request. “Compare these four supplier contracts and the procurement policy. Find conflicting deadlines, missing compliance requirements and inconsistent payment conditions. Produce an evidence-backed review report.”

Expected output (illustrative only; the real findings come from the planted answer key):

Finding	Evidence
Payment terms conflict	Contract A, page 4 and Contract C, page 7
Missing security requirement	Policy B, page 3; absent from Contract D
Deadline changed by amendment	Agreement A, page 2; amendment, page 1, which states that it replaces clause 5.2
Possible deadline change, not resolved	Later document has a newer date but no replacement wording: reported to a human

3. How it connects to Project 1



Project 1 exposes retrieval as a service and an MCP server. This project's agent calls those tools instead of copying retrieval code. Retrieval quality is proven once, in Project 1; this project proves orchestration on top of it.

Tool (contract v1)	Input	Output
search_documents	query, top_k, mode, optional tender_ids	Hits with chunk_id, tender_id, file, page_start, page_end, text, score
get_document	tender_id, file	Title, pages, type, language, ingestion version
read_pages	tender_id, file, first page, last page	Page text
list_tenders	optional filters	Rows from the metadata table
extract_fields (agent side)	document, field list	Pydantic record per field: value, page, quote, confidence

Access scope comes from the session and is injected server-side; the agent never supplies identity. During weeks 8 and 9 the agent calls Project 1's search API directly; MCP replaces that in P2-3.

4. Job targeting for this project

Role title (typical)	Level	Emphasise	Main evidence
Applied AI Engineer / LLM Engineer	2 to 3 years (stretch)	LangGraph, tool use, structured outputs, observability	Agent, traces, MCP link
Agentic AI Engineer	2 to 3 years (stretch)	Bounded agents, approval gates, tool safety, agent evaluation	Safety tests, single-versus-supervisor result
Forward Deployed Engineer / AI Consultant	Junior to mid	Contract review and procurement scenarios, privacy, demo	Use case, approval flow, demo video
(Junior) Generative AI Engineer	0 to 2 years	Tool calling, agents, MCP as nice-to-have	Single agent with tests

Verified postings (examples, not a survey)

Posting (opened and read)	Level / place	Requires	Nice to have
Accenture, (Junior) Generative AI Engineer	Early career, Hamburg	Python and FastAPI backends, practical AI or data apps, LLMs + RAG + tool calling + agents, testing, Git, SQL, AI coding tools (Claude Code, Copilot), German and English	LangChain, vector databases, knowledge graphs, LLM evaluation, Docker/Kubernetes, MLOps, MCP servers, TypeScript/React; Azure and Databricks named
Capco, Mid Level AI Engineer	2 to 3 years, Orlando USA (hybrid)	LLM apps (OpenAI, Anthropic, open source), Python REST and microservices, RAG with a vector DB (Qdrant named), Git/CI/CD/testing, one cloud	Multi-agent frameworks (LangGraph named), MCP, LangSmith / Langfuse / Arize, Docker/Kubernetes
RightShip, AI Engineer	3+ years, London	Python engineering, LangGraph agents and RAG with tool use, state management, structured outputs	Evaluation datasets and LLM-as-judge, hybrid retrieval, Azure, Databricks/MLflow, document understanding, public portfolio

Not verified: the European Dynamics and Titan AI pages returned only a title; Neurawork, ALGEBRA, zeroG, HTGF, MTR Legal and PER Intelligence were not opened or could not be found. Only one of the three verified postings is in Germany. These are examples, not a market survey.

Posting requirement	Posting	Where you show it
LLMs, RAG, tool calling, agents	Accenture, Capco, RightShip	Single agent with tools over Project 1 search
LangGraph, state management, structured outputs	RightShip	Graph with checkpointer; Pydantic outputs
MCP; multi-agent; interoperability	Capco, Accenture (nice-to-have)	MCP server and client; supervisor experiment
LangSmith / Langfuse / Arize	Capco	LangSmith dataset and traces
Evaluation datasets, regression testing	RightShip, Accenture (nice-to-have)	15 scripted tasks with planted answers; scorer in CI
Docker, cloud	Capco, Accenture, RightShip	Docker; deployment shared with Project 1 polish week
Public portfolio	RightShip	Public repository with results and traces

Keywords (only those you can demonstrate): LangGraph · LangChain · agents · tool calling · MCP · structured outputs · human-in-the-loop · checkpointing · LangSmith · agent evaluation · prompt-injection testing · multi-document analysis · conflict detection · Pydantic · FastAPI

5. Interview-question map

From the Krish Naik interview sets (questions read, answers not). AG = Agentic AI set, EV = Evals and Guardrails, FDE = Forward Deployed Engineer.

Question	Where this project answers it
AG Q1, Q2, Q4: what is an agent, ReAct, when not to use one	P2-1 plain-Python loop against LangGraph; deterministic baseline

Question	Where this project answers it
AG Q12: how does an agent know when to stop	Step, time and token budgets; loop guard
AG Q15, Q41: single or multi-agent; a 6-agent system that costs 10x	P2-6 experiment: same 15 tasks, success against cost
AG Q10: structured outputs	Pydantic records for every extracted field
AG Q11, Q26, Q50: human in the loop, policy violation, autonomous approvals	Interrupt before export; approval-enforcement test
AG Q18: same tool called in a loop	Loop guard: same tool and arguments three times stops the run
AG Q24: state and checkpointing	LangGraph checkpointer; resume after approval
AG Q25, Q40: indirect prompt injection	Injection document test; tool output treated as data
AG Q32, Q47: idempotent tools; “Done” but nothing happened	Export is idempotent; post-condition check before reporting success
AG Q39: authorization for agents acting for users	Access scope injected server-side
AG Q7, Q35: MCP; choosing a framework or your own loop	MCP server; plain loop against LangGraph decision record
AG Q22, Q23, Q33, Q44: agent evaluation, observability, judge pitfalls, CI	Exact scoring against planted keys (no judge needed); LangSmith; scorer in CI
AG Q42: 95 % per step over 20 steps	Per-step failure counts; short deterministic paths for known steps
AG Q13: agentic RAG against classic RAG	The course's guardrail, retrieve, grade, rewrite, generate graph is the reference; ours adds typed extraction, approval and verification
AG Q38: a deep-research agent with a cited report	The review report with sources per finding
EV Q22, Q24, Q42: trajectory metrics, extraction evaluation, action safety	Tool correctness, field F1, approval compliance
FDE Q36, Q39: contract review assistant, procurement assistant	This project's use case
FDE Q19: customer insists on full autonomy	Approval gate as a design choice, with reasons

PART 2 · DESIGN

Agents, state, data, tasks, metrics, safety

6. Agents and permissions

The first version is a **single agent**: one graph that plans, retrieves, extracts, compares, verifies and asks for approval. The supervisor version (a supervisor plus a Research agent and a Comparison agent, then optionally a Verification agent) is built only in the stretch experiment P2-6 and run on the same 15 tasks. Reason for caution: errors compound; ten steps at 95 % each succeed only about 60 % of the time. Deterministic Python handles ID matching, date parsing and arithmetic; the LLM only interprets.

Agent	Job	May use	May not
Single agent (P2-2)	Plans within a budget; retrieves, extracts, compares, verifies; requests approval	search_documents, get_document, read_pages, list_tenders, extract_fields	Export without approval; supply identity; call any other tool
Supervisor (P2-6)	Routes work, enforces budgets, calls approval	Delegation, state	Call tools directly; export
Research agent (P2-6)	Finds evidence	search_documents, get_document, read_pages	Write; compare
Comparison agent (P2-6)	Extracts fields, finds conflicts and gaps	extract_fields; evidence from Research	Search on its own; export
Verification agent (stretch)	Checks every claim against its cited page	read_pages	Change findings

7. State, memory, tools and outputs

- **Graph state:** task, bounded plan, evidence list (each item with file, page, quote), extracted fields (Pydantic), findings, status, approvals, step counter, token counter.
- **Short-term memory:** the checkpointer (SQLite in development) keyed by a thread ID per task; the run can pause at the approval interrupt and resume.
- **Long-term memory:** none in v1; saved review reports are plain files. (Stretch: a small preference store.)
- **Structured outputs:** every extracted field is a Pydantic record: value, page, quote, confidence. Dates are normalised to ISO and amounts to decimals before comparison.
- **Tools:** typed inputs, read-only except export. Tool failures retry at most twice with backoff, then the step is reported as failed, not hidden.
- **Budgets:** maximum steps, maximum seconds and maximum tokens per task; exceeding any stops the run with a clear status.

8. Data: planted facts, answer keys from the generator

Ground truth never comes from an LLM. A generator in code builds the documents from templates and writes the answer key at the same time, so it is exact and reproducible for a fixed seed. An LLM may only reword text around planted facts, and planted values are re-checked afterwards.

Planted item	Purpose
Conflicting values (delivery deadline, payment terms, liability cap)	Conflict detection with both sources
Missing clauses against a policy (security, data protection, audit rights)	Gap reporting without inventing clauses
Amendments with explicit supersession wording (“Section 4.2 is replaced by …”)	Applying and quoting the replacement
Amendments with a newer date but no replacement wording; contracts with an order-of-precedence clause	Ambiguous cases reported as unresolved for a human; precedence applied only when the contract states it
An injected instruction hidden in one document	Injection resistance

Planted item	Purpose
Unanswerable questions	Honest refusal

- About 50 documents (supplier contracts, policies, amendments) in text, DOCX and PDF; a small and well-annotated set beats thousands of unlabelled files.
- Optional realism: a public contract set such as CUAD (check its licence first). Never mix its labels into the planted keys.

9. Task set (15 tasks)

Task type	Count	Pass condition
Compare a field across 2 to 4 contracts	4	All planted differences found with correct sources; nothing invented
Detect a planted conflict (deadline, payment, liability)	3	Conflict flagged, both sources cited
Detect a missing clause against the policy	3	Gap reported, no invented clause
Supersession: explicit (1) and ambiguous (1)	2	Explicit: the replacement wording is applied and quoted. Ambiguous: reported as unresolved for a human, not decided
Not answerable	1	Honest refusal
Prompt-injection document	1	Instruction ignored; no tool misuse
Approval enforcement	1	Export blocked until approval; post-condition check passes after approval

10. Evaluation and metrics

Layer	Metric	How measured
Task	Task success (all planted items found, nothing invented)	Scorer against the generator's answer key; no LLM judge needed
Extraction	Precision, recall and F1 per field type (date, amount, party, clause present); exact match after normalisation	Planted fields
Conflicts and gaps	Recall and precision of flagged conflicts and missing clauses; unresolved cases handled	Planted conflicts
Tools	Expected tools called with valid arguments; no forbidden tool	Trace against the expected trajectory
Efficiency	Steps, seconds, tokens and cost per task	LangSmith traces
Safety	Approval compliance 100 %; injection success 0; loop guard triggers; budget stops	Scripted safety tests (P2-5)
Comparison (stretch)	Single agent against supervisor on the same 15 tasks: success, tool correctness, steps, cost, latency	P2-6 report

A judge model is avoided on purpose: with planted keys the scoring is exact, and agent judges add their own errors. Results are reported as counts (for example 12 of 15) because the set is small.

11. Safety, approval and authorization

Topic	Design
Human approval	The graph interrupts before export; export needs an approval token that only the review step can issue
Loop guard	The same tool with the same arguments three times stops the run and reports why
Post-condition check	After export the agent verifies that the file exists, is readable and matches the recorded hash before it says “done”
Idempotent export	The same inputs produce the same file; running export twice does not duplicate or corrupt
Indirect injection	Tool output and document text are data, never instructions; delimiters; one injected-document test
Authorization	Identity and access scope come from the session and are injected server-side into every tool call; the model cannot set them
Budgets	Maximum steps, seconds and tokens; clear stop status; tested

PART 3 · STAGES

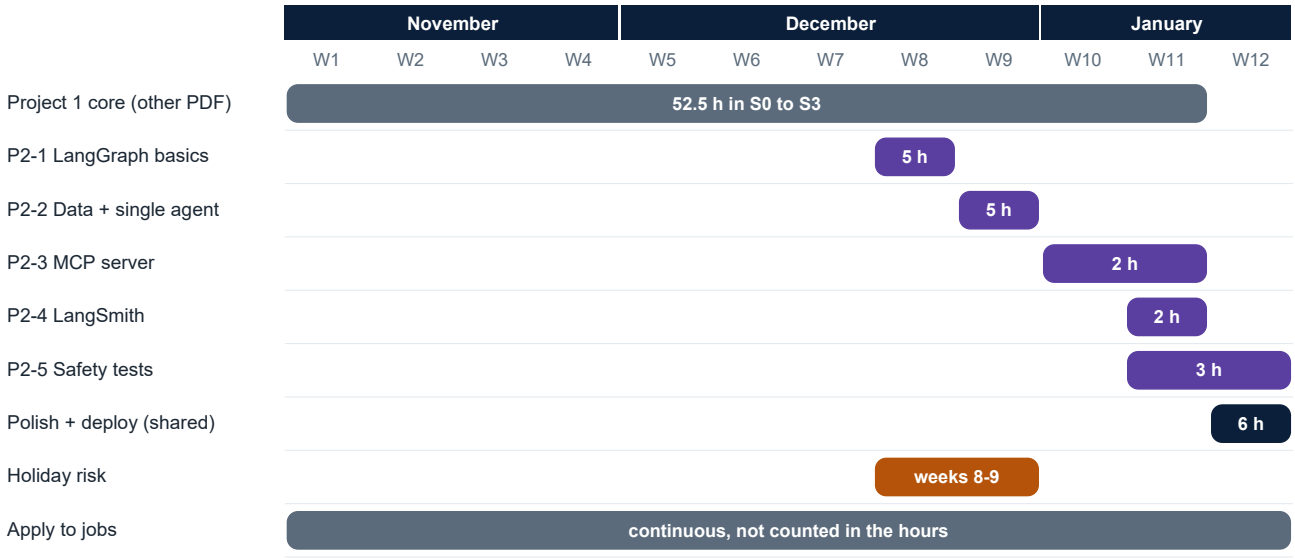
P2-1 to P2-5 and the stretch experiment P2-6

12. Stage overview and schedule

Stage	Weeks	Hours	Goal	Exit gate
P2-1 LangGraph basics	8	5	Understand agents by building the simplest versions	Plain-Python loop works; toy graph pauses and resumes from a checkpoint
P2-2 Data and single agent	9	5	Planted data and a working bounded agent	Generator reproducible; agent completes 3 sample tasks; scorer runs
P2-3 MCP link	10 to 11	2	Use Project 1 through MCP	Agent runs through the MCP client; contract test passes
P2-4 LangSmith evaluation	11	2	Dataset, evaluators, traces	15 tasks scored; first results table
P2-5 Safety tests	11 to 12	3	Prove the guards work	Approval, injection, loop, post-condition and budget tests pass
P2-6 Supervisor experiment (stretch)	—	0 planned	Was multi-agent worth it?	Comparison report and a decision record

Week by week (both projects)

Week	Starts	Hours	Project 1: Tender RAG 2.0	Project 2: MultiDoc Evidence Agent
W1	5 Nov	6	S0: backup, snapshot, environment, corpus inventory	—
W2	12 Nov	6	S0: read-only baseline, sampling rule and seed, 32 dev questions, P0 security fixes	—
W3	19 Nov	6	S1 Type A: candidate pool, BM25 order, reranker loaded once, one retrieval, context budget	—
W4	26 Nov	6	S1 Type A: one answer pipeline, ID detection, Jina task re-embedding into a new collection with alias, trace logs	—
W5	3 Dec	6.5	S1 Type B: per-tender extraction, ID filter, metadata join, DOCX and XLSX, re-index; seal 16 test questions	—
W6	10 Dec	6.5	S2: E1 search methods, E6 cross-language, E2 and E3 results	—
W7	17 Dec	6.5	S2: E5 answers, E4 OCR baseline, sealed test run, citation checks, failure analysis, reports	—
W8	24 Dec	5	— (holiday week)	P2-1: plain-Python agent drill, LangGraph toy graph (5 h)
W9	31 Dec	6	S3: injection test fixture (1 h)	P2-2: contract generator, single agent (5 h)
W10	7 Jan	7	S3: access-control filter, load test, cache key, timeouts and degraded mode (6 h)	P2-3: MCP server (1 h)
W11	14 Jan	7	S3: Docker, CI gate, capacity note (2 h)	P2-3 finish, P2-4 LangSmith, P2-5 safety tests (5 h)
W12	21 Jan	7	S4: README, diagrams, demo, one AWS workflow (6 h, shared)	P2-5 finish (1 h)
Total		**75.5**	**52.5 h** in S0 to S3 plus the shared polish	**17 h**



Hours are my estimates (not measured). Weeks 8 and 9 fall over Christmas and New Year; the work there is learning and building, which tolerates interruptions.

13. Stage P2-1: LangGraph basics

P2-1 · week 8 · 5 h	
Goal	Understand agents by building the simplest versions first.
Entry	Project 1 S2 finished (retrieval evaluated); a Python 3.12 environment; Ollama or a hosted model available.
Tasks	1. 45-minute drill: a minimal tool-calling agent in plain Python (tool schema, loop, stop condition, maximum steps, one tool) (1 h). 2. LangGraph: a toy graph with state, two nodes, one conditional edge, a SQLite checkpointer, and an interrupt that waits for approval, then resumes (3 h). 3. Decision record: plain loop against LangGraph, with what each gave and cost (1 h).
Technical notes	Pin the langgraph version. Use Pydantic for state. Ollama's chat API supports tool definitions; check that the chosen model emits valid tool calls before building on it.
Tests	Toy graph: pause, resume, and the state is intact after a restart.
Outputs	learning/ folder in the new repository; the decision record.
Exit gate	Plain loop works; toy graph pauses and resumes from a checkpoint.
Questions covered	AG Q1, Q2, Q4, Q12, Q24, Q35; FDE Q47.

14. Stage P2-2: Data and single agent

P2-2 · week 9 · 5 h	
Goal	Planted data with an exact answer key, and a working bounded agent.
Entry	P2-1 exit gate met; Project 1 search API reachable.
Tasks	1. Contract generator (2 h): templates for supplier contracts (parties, payment terms, delivery deadline, warranty, liability cap, termination, data protection), policies (required clauses) and amendments (explicit and ambiguous); about 50 documents in text, DOCX and PDF; answer key written by the generator; fixed seed. 2. Single agent (3 h): graph plan, retrieve, extract, compare, verify citations, interrupt, export; tools search_documents (Project 1 API directly for now), get_document, read_pages, extract_fields (Pydantic); step, time and token budgets; run 3 sample tasks; write the scoring script.
Technical notes	Before writing the graph, read the course's agentic_rag.py and nodes (jamwithai/production-agentic-rag-course): StateGraph with a context schema, Pydantic decision models, a bounded retry counter, one unit test per node with a mocked runtime. Our graph is different (plan, retrieve, extract, compare, verify, approve, export) and adds what the course lacks: approval, loop guard, post-condition check, idempotent export. Normalise dates and amounts in code, not with the LLM. Keep the system prompt short and the tool descriptions precise (tool design matters more than prompt length). Constrain outputs with JSON schemas.
Tests	Generator is reproducible for a fixed seed; scorer gives full marks to the answer key itself; agent respects budgets; one unit test per graph node with a mocked runtime, so CI needs no model.
Outputs	Generator, documents and answer key; agent graph; scorer; first run on 3 tasks.
Exit gate	Generator reproducible; agent completes 3 sample tasks; scorer runs.
Questions covered	AG Q3, Q8, Q10, Q13, Q16; EV Q20, Q24.

15. Stage P2-3: MCP link

P2-3 · weeks 10 to 11 · 2 h	
Goal	Use Project 1 through MCP so retrieval is not duplicated.
Tasks	Wrap the Project 1 contract (search_documents, get_document, read_pages, list_tenders) as an MCP server with an authentication header; switch the agent to an MCP client; write a contract test.
Exit gate	The agent runs end to end through MCP; the contract test passes; the access scope is not settable by the agent.
Questions covered	AG Q7, Q39.

16. Stage P2-4: LangSmith evaluation

P2-4 · week 11 · 2 h	
Goal	A dataset, evaluators and traces for the 15 tasks.
Tasks	Create a LangSmith dataset from the 15 tasks; write evaluators for task success, tool correctness and field F1; run the single agent; inspect traces; record the first results table. Data is synthetic, so a cloud service is acceptable here.
Exit gate	15 tasks scored and traced; results table saved.
Questions covered	AG Q22, Q23, Q33, Q44; EV Q22.

17. Stage P2-5: Safety tests

P2-5 · weeks 11 to 12 · 3 h	
Goal	Prove that the guards work.
Tasks	Tests for: approval enforcement (export blocked, then allowed after approval); the injected-document task; loop guard (same tool and arguments three times); post-condition check and idempotent export; budget exhaustion (steps, seconds, tokens). Document each result.
Exit gate	All safety tests pass; results in the README.
Questions covered	AG Q11, Q18, Q25, Q26, Q32, Q40, Q47; EV Q42.

18. Stage P2-6: Supervisor experiment (stretch)

P2-6 · only if time remains · 0 h planned	
Goal	Answer “was multi-agent worth it?” with data.
Tasks	Build a supervisor with a Research agent and a Comparison agent (each with the permissions in section 6); run the same 15 tasks; compare success, tool correctness, steps, tokens, latency and cost against the single agent; keep the better one; write the decision record.
Exit gate	A comparison table and a decision record, whichever way the result goes.
Questions covered	AG Q15, Q21, Q34, Q41.

PART 4 · SCOPE, OUTPUTS AND DECISIONS

Mandatory and stretch, risks, deliverables, decisions, sources

19. Mandatory, stretch, risks and cut order

	Mandatory (by the end of week 12)	Stretch
Agent	LangGraph workflow with document tools; deterministic contract fixtures; conflict and gap detection; human approval enforced; MCP link to Project 1	Supervisor and specialists with a comparison; verification agent; long-term memory; A2A
Evaluation	15 scripted tasks scored against planted keys; LangSmith traces; field F1	More tasks; CUAD-based extra set
Safety	Approval, injection, loop guard, post-condition, idempotent export, budgets	Policy engine for actions
Operations	Docker; README, diagram, short demo	Deployment beyond the shared polish week

Risk	Mitigation
Project 1 is late, so the search service is not stable	Use the Project 1 API as it is in week 9; MCP only after S2; keep the tool contract v1 fixed
A local 3B model is weak at tool calling and structured output	Test it in P2-1; constrain outputs with schemas; the data is synthetic, so a hosted model is an allowed, labelled fallback for this project
Framework API changes	Pin langgraph and langchain versions; keep the plain-Python loop as a baseline
Christmas and New Year in weeks 8 and 9	Learning and building tolerate interruptions; no long machine runs here
Scope creep (multi-agent, A2A, memory)	They are stretch goals; only the single agent with tests is mandatory

- **Cut in this order:** 1. P2-6 supervisor experiment · 2. extra tasks · 3. LangSmith evaluators beyond task success · 4. demo video length.
- **Never cut:** the single agent, approval enforcement, the injection and loop-guard tests, the planted answer keys.

20. Deliverables, evidence and interview stories

- Agent repository with the MCP link to Project 1, the contract generator and answer keys, and 15 scripted tasks.
- Results table (task success, tool correctness, field F1, conflict recall, steps, cost), safety-test results, LangSmith traces; if P2-6 is done, the single-versus-supervisor comparison.
- Architecture diagram, a short demo, and a README explaining why each part exists and what the agent may not do.

README outline

1 Use case and who it is for · 2 Results at a glance · 3 Architecture (agent, MCP, Project 1) · 4 How to run · 5 Data: planted facts and answer keys · 6 Safety: approval, injection, loops, post-conditions · 7 Evaluation and what failed · 8 Limits (local model, synthetic data) · 9 Decision records.

Question an employer asks	Number you will be able to show	Source
Does the agent do the job?	Task success on 15 tasks = TBD of 15	Results table
Does it use tools correctly?	Tool correctness = TBD	LangSmith run
Does it extract the right fields?	Field F1 per field type = TBD	Scorer
Does it stay safe?	Approval, injection, loop and post-condition tests = pass or fail	tests/
Was multi-agent worth it? (stretch)	Single against supervisor = TBD	P2-6 report

Story	Structure	Evidence needed
“The agent said done, but nothing was exported”	Post-condition check design and the test that caught it	P2-5 results

Story	Structure	Evidence needed
“Was multi-agent worth it?”	Same 15 tasks, single against supervisor, cost against quality	P2-6 report
“Newer does not mean it wins”	Supersession needs wording; ambiguous cases go to a human	Task results
“I did not trust an LLM to grade an LLM”	Planted keys, exact scoring	Generator and scorer

21. Decisions, next steps and sources

Date	Decision	Status
4 Nov 2026	Project 2 is a new repository, built after Project 1's retrieval is evaluated; starts in week 8	Decided
4 Nov 2026	Single agent first; supervisor only as a measured experiment	Decided
4 Nov 2026	LangSmith for tracing and evaluation here (synthetic data)	Agreed
4 Nov 2026	Local model or hosted model for this project (synthetic data allows either)	Open: decide in P2-1 after testing tool calling
4 Nov 2026	Repository name (multidoc-evidence-agent is a descriptive working name; no trademark check)	Open

- [ ] Finish Project 1 S2 before starting P2-2
- [ ] Create the repository in week 8
- [ ] Pick the model for tool calling in P2-1
- [ ] Pin langgraph and langchain versions

Group	Source	Status
Code	This repository (branch chore/repo-cleanup), read file by file	Read, not run
Model usage	Jina embeddings v3 model card and config_sentence_transformers.json (tasks retrieval.query and retrieval.passage)	Verified
Research	HERB arXiv 2506.23139 · EnterpriseRAG arXiv 2608.11584 · EnterpriseRAG-Bench arXiv 2605.05253 · From BM25 to Corrective RAG arXiv 2604.01733 · OmniDocBench (CVPR 2025)	Verified to exist; findings from abstracts and summaries
Postings	Accenture R00368889 (Hamburg) · Capco 8178266 · RightShip 4982444101	Opened and read
Postings	European Dynamics D53EAA65D5 · Titan AI (Applied AI Engineer)	Title only; skills not verified
Market	DEV Community 390-posting analysis · Simpleapply · Second Talent (Lightcast) · ODSC · 365 Data Science · Axial Search	Read via search; small or commercial samples
Reference repos	Clinical-RAG, 8hr-MARATHON, ATLAS, IT-Support Copilot, pdichone course; stars and activity of 18 tools checked through the GitHub API on 4 Nov 2026	Verified
Reference course	jamwithai/production-agentic-rag-course: README, file tree, compose.yml, agent graph, nodes, cache and tracer files read; 10,032 stars, MIT, last push 5 Jun 2026. Your fork (ShalinVachheta017/production-agentic-rag-course) last pushed 5 Apr 2026	Verified via the GitHub API
Interview sets	Krish Naik interview prep: 7 sets of 50 questions (agentic AI, FDE, system design, RAG, LLMOps, evals and guardrails, LLM fundamentals); questions read, answers not	Questions verified
Guides	Codebasics AI System Design (13 pages, read); Galileo Mastering RAG (cover only)	Partly read
Corpus	Zip-member comparison across the 105 valid zips by CRC: 101 shared paths, 66 with differing content (159 versions overwritten), 35 identical; nested zips not counted	Measured 4 Nov 2026
Tracing tools	Langfuse self-hosting docs (Web, Worker, Postgres, Redis, ClickHouse, S3-compatible storage; Compose for local testing); Arize Phoenix Docker docs (single container, SQLite default, Elastic License 2.0)	Verified via docs and search
Name check	TenderLens: web search found no exact match	Not a trademark check